

שאלות ראיון

JavaScript שאלות

(1) מה ההבדל בין var, let ו-const?

תשובה : אז ראשית נבדיל בין const לאחרים כי ההבדל פשוט יותר :
const הוא בעצם משתנה קבוע , הכוונה שכבר בשלב ההצהרה חייב לאתחל אותו בערך, ובשום שלב בתוכנית אסור לשנות אותו (לא ניתן) .

וההבדל בין השניים האחרים:
בשניהם טיפוס המשתנה נקבע רק בשלב הריצה לפי ההקשר, כמובן שאין צורך לאתחל בשלב ההצהרה.
בעבר השתמשו ב var והיום נפוץ יותר להשתמש ב let , ההבדל העיקרי בא לידי ביטוי ב scope , כלומר let אינו מוכר מחוץ לscope שהוגדר בניגוד ל var שכן מוכר .
דוגמת קוד :

```
if ( true )  
{  
  let a = 10  
  var b = 20  
}  
console.log(a)// undefined  
console.log(b)// יודפס 20 תקין
```

(2) מה זה hoisting ואיך הוא עובד?

זה בעצם האפשרות להשתמש במשתני var (בלבד) לפני שהם הוגדרו, הערך בשימוש זה יהיה undefined.
מאחורי הקלעים ההגדרות והצהרות המשתנים (בלבד, ללא האתחול) מורמים לראש ה scope כך שבקריאה לאותו משתנה לא תיזרק שגיאה , לדוג :

```
console.log("x is " + x) // יודפס undefined  
var x = 8  
console.log("x is " + x) // פה כבר יודפס 8  
" var x " בפועל הוא מסתכל על זה כאילו שבשורה הראשונה כתוב :
```

(3) מה ההבדל בין == ל-===?

ההבדל הוא בסוג ההשוואה, שבעצם === בודק האם הערכים עצמם שווים ובנוסף האם הם מאותו הסוג (number, string וכו). לעומת == שבדוק רק האם הערכים עצמם שווים (בעזרת המרת המספר למחרוזת והשוואה) ואם הם אינם מאותו טיפוס עדיין מחזיר "אמת" עבור אותה השוואה.
לדוג :

```
let a= 5  
let b= '5'  
  
return ( a == b ) // true הערכים שווים  
return ( a === b ) //false לא מאותו סוג
```

4) מה זה closure ותני דוגמה?

כאשר יש לי פונקציה שמקבלת משתנה (מכל סוג שהוא) ומחזירה פונקציה, אז כשאני קוראת לפונקציה הזו ושולחת ערך אז זה הערך שנשמר עבור הפונקציה שחזרה, וכל פעם כשאני משתמשת בפונקציה שחזרה לי אז הערך הזה נשמר (מהפעם הראשונה), וניתן להשתמש בו במידת הצורך. הערך הזה נקרא closure. לדוג:

```
function a (b){  
  return (x) => { b + x }  
}  
  
const dummy = a(5) // closure = 5  
  
let result1 = dummy(4) // result1 = 5 + 4 = 9  
  
let result2 = dummy(5) // result2 = 5 + 5 = 10
```

5) איך עובד the event loop?

בכל פעם שיש לי פונקצית callback היא נכנסת לתור של ה event loop וברגע שהיא צריכה להתבצע (בין אם נגמר הטיימר לבין אם הופעל ה event) אז היא יוצאת מהתור ומתבצעת. אלו בעצם סוג של listeners שמקשיבים לאירועים או לשעונים (timers)

6) מה ההבדל בין null ל-undefined?

משתנה שהוא undefined הוא פשוט לא מוגדר. כלומר עדיין אין ערך בתוכו אבל שמור לו מקום מסוים בזיכרון וניתן להדפיס אותו לקונסול ללא שגיאת ריצה, לעומת null שאינו קיים בזיכרון, כלומר אין הפנייה לשום מקום בזיכרון, וכל פעולה שעושים עליו תגרום לשגיאת ריצה ונפילת התוכנית

7) מה זה Promise ומה מצביו האפשריים?

כשאני מבצעת קריאה כל שהיא לשרת, השרת מחזיר לי את ה data כ Promise, שכדי להשיג את הנתונים עצמם מתגובה זו אני צריכה להפעיל פונקציות שמוגדרות להחזיר נתונים שאוכל לעבוד איתם, לדוגמה (then(data): בתוך ה data יש בעתם רשימה הוא אובייקט שאני יכולה לעבוד איתו ב javascript. catch(error): אם הבקשה לשרת נכשלה מכל סיבה שהיא אז catch תופסת אותו ומטפלת בהתאם ויש finally וכו

8) מה ההבדל בין synchronous ל-asynchronous?

אז synchronous זה שסדר ביצוע הקוד הוא לפי סדר השורות שהוא כתוב, כלומר זה מתבצע שורה אחר שורה, ורק לאחר שביצוע שורה אחת מסתיים רק אז הדורה הבאה אחריה מתבצעת וכך הלאה. וב asynchronous סדר ביצוע הפעולות אינו לפי סדר כתיבתן אלא לפי איך שאני מגדירה כל שורה, אם עבר שורה מסוימת הגדרתי אותה כ async אז אנחנו לא מחכים שהיא תסתיים להתבצע, אלא שומרים סימן מסוים בצד וממשיכים הלאה, ולפי הסימן היא אנחנו חוזרים אליה מידת הצורך, לפי ההקשר וכו

9) מה זה destructuring ותני 2 דוגמאות?

זה תחביר JS שמאפשר לפרק מערכים, אובייקטים ואוספים מסוימים למשתנים, זה יכול גם להגדיר סדר מיקומים למערך או לשמור מקומות ספציפיים במערך וכל השאר נכנס רגיל.
דוגמה:

```
const [a, b] = [5, 7] // a = 5, b = 7  
[a, b, ...rest] = [10, 20, 30, 40, 50]; // a = 10, b = 20, rest = [30, 40, 50]
```

```
const arr = [1, 2, 3]  
const [a, b, c] = arr // a = 1, b = 2, c = 3
```

10) מה זה spread operator ובמה הוא שונה מ-rest?

הוא "פורס" אוסף ערכים למרכיבים בודדים.
זה שאפשר להעתיק אוספים לאוספים חדשים בעזרת 3 נקודות (. . .)
יש 3 שימושים נפוצים:

פרמטר רשימה למערך: myFunction(a, ...iterableObj, b)
מערכים: [...iterableObj, '4', 'five', 6, 1]
אובייקטים: { key:"value", ...obj }

ו rest עושה בדיוק הפוך הוא אוסף מספר ערכים למשתנה אוסף אחד:

```
function sum(...numbers) {  
  return numbers.reduce((acc, n) => acc + n, 0);  
} // numbers = [1, 2, 3, 4]
```

שאלות React

- (1) מה זה Virtual DOM ולמה הוא קיים?
DOM הוא ראשי תיבות Document Object Model , זה מודל שמייצג דף אינטרנט שכתוב בשפת תכנות כלשהי, במבנה של מסמך (HTML לדוג) באמצעות עץ וכל ענף בעץ מסתיים בצומת שמכילה אובייקט,
זה בעצם מאפשר גישה לאובייקטים האלו שניתן לשנות את המבנה והתכונות והפעילות של הדף באמצעותם.
וה Virtual DOM זה בעצם עותק וירטואלי של הDOM האמיתי והוא כמו המטמון , שכל גישה ל DOM המקורי היא מאוד יקרה ולא על כל שינוי קטן אנחנו רוצים לבצע אותה (לעדכן את כל הדף) , אז אנחנו משתמשים ב DOM הוירטואלי שבכל שינוי יוצרים גרסה חדשה של הוירטואל DOM ומשווים אותו לגרסה הישנה ורק את השינויים מעדכנים ב DOM המקורי
- (2) מה ההבדל בין props ל-state?
ה props זה שמעבירים מידע בין רכיבים לרכיבים בנים כפרמטר , ניתן גם להגדיר שלמידע שנכתב ברכיב מסוים יהיה גישה לכל הרכיבים בעץ מתחתיו , ולא ניתן לשנות אותו ברכיבי הבנים .
ו state זה המידע ששמור על אותו רכיב ונשאר בתוכו , ואין גישה מרכיב אחר, וניתן ורצוי לשנות אותו כדי שהמידע יהיה מעודכן ועקבי . וכאשר ה state מתעדכן, הרכיב מבצע render מחדש.
- (3) מה זה useEffect ומתי הוא רץ?
כשאני לא רוצה שפעולה מסוימת תקרה / תופעל בכל רינדור מחדש אז אני משתמשת ב useEffect שמאפשר לי לשלוט כמה ומתי ביחס לרינדור הדף , תופעל הפעולה שאכניס לתוכו , בעזרת פרמטר נוסף שמקבל , שאומר לו בעצם מתי לרוץ .
משתמשים בו בעיקר בשביל פעולות חיצוניות כמו בקשות API, טיימרים, גישה ל DOM או event listeners.
- (4) מה זה dependency array ב-useEffect ולמה הוא חשוב?
זה בעצם הפרמטר השני שדיברנו עליו קודם , והוא מערך תלויות שלפי התלות שאני כותבת בתוכו הuseEffect ירוץ .
זה יכול להיות משתנה [value] , שאומר בכל פעם שהוא ישתנה ה useEffect יופעל,
וזה יכול להיות רשימה ריקה [] , שאומרת תופעל פעם אחת בטעינת הרכיב,
אם לא שולחים כלום זה ירוץ אחרי כל רינדור.
- (5) למה צריך key ברשימות וכיצד לבחור אותו?
כדי שיהיה אפשר להבדיל בין הרשומות באופן ייחודי , ושיהיה אפשר לחפש ולגשת לאיבר מסוים , ללא המפתח לא ניתן יהיה לזהות או להבדיל בין הערכים כי ייווצרו מספר איברים זהים בדיוק וזה יוצר בעיות .
את ה key בוחרים לפי ID או עם משתנה רץ או HASH וכו
- (6) מה זה controlled component מול uncontrolled component?
ניתן להגדיר רכיב controlled שהוא בעצם נשלט על ידי רכיב האב שלו , הכוונה שכל הפונקציונליות והמידע והכל, רכיה האב מנהל , אין לו state משלו וכו , זה כן יוצר הרבה עבודה לאבא שהוא צריך ממש לטפל בו ולתפעל אותו , אבל מצד שני מתאפשרת לו גישה נורא מהירה ונוחה לבנו.
וכשרכיב הוא uncontrolled זה בעצם אומר שאף אחד לא שולט בו , הוא אדון לעצמו ויש בו state משלו והוא פועל מעצמו , זה נחשב נוח לרכיב האב שלו כי הוא פחות צריך להתעסק בו ולהפעיל אותו , אבל כשהוא כן רוצה לגשת אליו או לקבל ממנו נתונים זה כבר יותר מסובך ובעייתי לו .

(7) איך מטפלים בצורה נכונה ב-loading ו-error states?

בעזרת Hooks שמגדירים עבור loading ו-error states ובפונקציה שמבצעים פעולה בדר"כ הקשורה בנתונים בעצם מעדכנים את המשתנים loading, error, שבתחילה מאתחלים ב null, false, בעזרת setLoading, setError ל true ואם יש שגיאה אז בהודעת שגיאה, וב return בודקים האם המשתנים אמת או שקר ל loading או null ל error ולפי זה מציגים מה שצריך בהתאם

(8) מה זה useState ולמה הוא לא מעדכן מיד?

זה בעצם מאפשר לשמור ולעדכן מצב - state של רכיב בצורה יעילה, הוא לא מתעדכן מיד כיוון שהוא קודם כל הוא מחכה שכל הטיפול באירועים יסתיים ורק אז הוא מעדכן את המצב וזה בעצם מונע הרבה רינדורים מיותרים, כי בכל פעם שהמצב משתנה מתבצע רינדור

(9) מה זה lifting state up?

נכון לעכשיו ל state שייך לרכיב שבו הוא מוגדר, אך אם בכל זאת צריכים לדעת את המצב state בין הרכיבים, אז אפשר לעשות את זה באמצעות הגדרת מצב כולל ברכיב אבא שעוטף את כל הרכיבים שרוצים לחלוק מצב, ולהעביר את זה כ props בפרמטר לבנים, ואפשר גם לאפשר תקשורת בין הבנים עצמם דרך ההורה, בעזרת הצהרה על מצב משותף ברכיב ההורה.

(10) מה זה custom hook ומתי שווה להשתמש בו?

זה בעצם שאני כותבת hook משלי שאני רוצה להשתמש בו בכמה רכיבים בלי לכתוב את אותו הקוד בכל רכיב. זה בעצם כמו ה - useState, useEffect, רק שהוא לא מובנה ב react אלא אני כותבת אותו, לפי הצרכים שלי. לדוגמא אם אני רוצה ברכיב מסוים לדעת על מצב החיבור לרשת אז אני צריכה להגדיר משהו כזה:

```
const [online, setonline] = useState(true)
```

ולכתוב את זה בכל הרכיבים שארצה לבדוק לגבי חיבור רשת, אז אני פשוט יכולה להגדיר Hooks משלי שיעשה את זה (באופן גלובלי) ופשוט לייבא את זה ברכיבים שאני רוצה, לדוג: ...import {UseOnline} from